Latest updates: https://dl.acm.org/doi/10.1145/3659994.3660313

RESEARCH-ARTICLE

# Towards Energy-Aware Execution and Offloading of Serverless Functions

**CECILIA CALAVARO**, Tor Vergata University of Rome, Rome, RM, Italy

**GABRIELE RUSSO RUSSO**, Tor Vergata University of Rome, Rome, RM, Italy

**MARTINA SALVATI**, Tor Vergata University of Rome, Rome, RM, Italy

**VALERIA CARDELLINI**, Tor Vergata University of Rome, Rome, RM, Italy

**FRANCESCO LO PRESTI**, Tor Vergata University of Rome, Rome, RM, Italy

**Open Access Support** provided by:

**Tor Vergata University of Rome**

**PDF Download**
**3659994.3660313.pdf**
**02 March 2026**
**Total Citations:** 6
**Total Downloads:** 468

**Published:** 29 July 2024

**Citation in BibTeX format**

FRAME '24: 4th Workshop on Flexible Resource and Application Management on the Edge
*June 3 - 7, 2024*
*Pisa, Italy*

**Conference Sponsors:**
SIGARCH

# Towards Energy-Aware Execution and Offloading of Serverless Functions

Cecilia Calavaro
calavaro@ing.uniroma2.it
Tor Vergata University of Rome
Italy

Gabriele Russo Russo
russo.russo@ing.uniroma2.it
Tor Vergata University of Rome
Italy

Martina Salvati
martina.salvati@alumni.uniroma2.eu
Tor Vergata University of Rome
Italy

Valeria Cardellini
cardellini@ing.uniroma2.it
Tor Vergata University of Rome
Italy

Francesco Lo Presti
lopresti@info.uniroma2.it
Tor Vergata University of Rome
Italy

## ABSTRACT

The convergence of Function-as-a-Service (FaaS) with Edge computing presents a promising avenue to address the demands of geo-distributed and pervasive applications. Yet, leveraging both the reduced latency of Edge and the scalability of FaaS necessitates novel architectures and implementations, given the higher heterogeneity and resource constraints of Edge environments. An important issue regards energy management, as serverless functions may be executed in energy-constrained computing nodes (e.g., powered by batteries or renewable yet uncertain energy supplies).

In this paper, we present a solution for energy-aware function execution relying on two key mechanisms: (i) the ability of choosing among multiple function implementations at run-time, trading off energy savings with approximate computation results; (ii) computational offloading, to move function execution from the Edge to the Cloud, based on energy-aware policies. We integrate our solution in Serverledge, an open-source FaaS platform designed for the Edge-to-Cloud continuum. Our evaluation shows that energy-aware offloading and execution are effective in extending the lifespan and throughput of battery-powered nodes.

## CCS CONCEPTS

• **Computer systems organization → Cloud computing**.

## KEYWORDS

Serverless computing, Functions as a Service, Energy management

**ACM Reference Format:**
Cecilia Calavaro, Gabriele Russo Russo, Martina Salvati, Valeria Cardellini, and Francesco Lo Presti. 2024. Towards Energy-Aware Execution and Offloading of Serverless Functions. In *Workshop on Flexible Resource and Application Management on the Edge (FRAME '24), June 3–7, 2024, Pisa, Italy*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3659994.3660313

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*FRAME '24, June 3–7, 2024, Pisa, Italy*
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0641-7/24/06…$15.00
https://doi.org/10.1145/3659994.3660313

## 1 INTRODUCTION

Serverless computing and the Function-as-a-Service (FaaS) paradigm have enjoyed growing popularity over the last years [14]. Seamless scalability, simplified operations, and flexible pricing are among the most attractive and distinguishing features of FaaS that contributed to its popularity. While FaaS has been initially adopted and developed in the context of cloud computing (e.g., as part of commercial offerings like AWS Lambda), with the emergence of Edge computing, researchers and practitioners have shown increasing interest in adopting FaaS at the edge of the network as well [13, 19, 29].

However, the convergence of this new paradigm and Edge computing brings along its own set of challenges, mostly because of the resource constraints and heterogeneity of Edge environments [19, 22]. Moreover, the issues related to energy consumption play an increasingly important role in the design and development of distributed computing systems, including serverless computing systems [26]. Such issues are particularly relevant for serverless functions possibly executed in energy-constrained edge devices (e.g., powered by batteries or renewable yet uncertain energy supplies).

So far, research on serverless computing has mostly focused on other relevant metrics revolving around performance and monetary costs, with few works coping with sustainability aspects. Recently, researchers have started investigating solutions to bring energy efficiency and awareness into serverless systems. Most the approaches so far focused on techniques to schedule invocation requests to the most suitable FaaS node accounting for energy-related metrics (e.g., [2, 12]), or exploiting hardware-specific features to increase energy efficiency (e.g., [4]).

In this paper, we present an approach for energy-aware function execution in a FaaS system deployed over the Edge-to-Cloud continuum. Specifically, we leverage two key ideas, namely approximate computing and computational offloading, to reduce the energy consumption in energy-constrained FaaS devices. First, the proposed approach considers multiple implementations (or, configurations) of each function, corresponding to different trade-offs between resource demand, energy consumption and execution accuracy. For instance, a serverless function implementing a prediction task might use different ML models to trade-off energy demand with

accuracy. Our solution aims to dynamically select the most suitable function implementation to use at run-time, based on current working conditions.

As a second fundamental mechanism, the proposed approach relies on computational offloading to move function execution from the Edge to the Cloud when needed, to reduce energy consumption in the Edge devices. The two mechanisms are controlled by an energy-aware policy, which aims to extend the lifespan of battery-powered FaaS nodes. We integrate our solution in Serverledge [24], an open-source FaaS platform proposed by our group for the Edge-to-Cloud continuum.

Our key contributions can be summarized as follows:

- We extend the architecture of Serverledge to introduce energy monitoring and management components.
- We design and implement a mechanism for run-time selection of function implementations to trade-off energy consumption with computation accuracy.
- We devise a proof-of-concept energy-aware policy to extend the lifespan of battery-powered FaaS nodes, exploiting computational offloading and multiple function implementations.
- Prototype experiments show that energy-aware offloading and execution are effective in extending the lifespan and throughput of battery-powered nodes.

The remainder of this manuscript is organized as follows. We provide background information about FaaS and Serverledge in Sec. 2. We describe the proposed approach in Sec. 3 and evaluate it in Sec. 4. We review related work in Sec. 5 and conclude in Sec. 6.

## 2 BACKGROUND

Serverless computing is a computing paradigm where users focus solely on writing and deploying application code, while the provider takes care of all operational aspects, including resource allocation and scaling. A fine-grained pay-as-you-go cost model is adopted, with users being billed only for the resource usage due to application execution. As such, users incur no cost when application is idle and they do not need to predetermine the amount of compute resources necessary for their workload [25].

Within the serverless computing landscape, the Function-as-a-Service (FaaS) paradigm emerged as the most notable model. It enables users to deploy and execute units of computation, consisting of *functions*, and be charged only when they are executed. FaaS operates on an event-driven processing model, where code is packed in ephemeral containers, allocated as needed in response to events or requests.

FaaS has initially gained traction as part of commercial cloud offerings (e.g., AWS Lambda, Google Cloud Functions, Microsoft Azure Functions) and open-source frameworks (e.g., Apache OpenWhisk, OpenFaaS). Recently, more options have appeared to support FaaS at the edge of the network as well (e.g., faasm [27], Sledge [9], Serverledge [24]).

Compared to traditional deployments based on virtual machines (VMs) or containers, where resources must be allocated upfront to accommodate incoming workloads, serverless and FaaS enable significant cost savings during periods of inactivity (e.g., in presence of intermittent workloads). Such cost savings are potentially associated with energy savings as well, with potential reduction of environmental impact thanks to higher levels of resource sharing within the provider infrastructure.

As charge is based on the reservation of computing resources, data centers - as cloud and serverless computing themselves - do have a considerable impact on the energy and power wordlwide requests. Estimates suggest a range from 200 to 500 terawatt-hours (TWh), equivalent to 1–2.5% of the world's total energy consumption [17]. Moreover, this quantity is expected to grow with the rising demand for cloud computing, as in 2006 the number of machines in data centers were 11 million, compared to the 18 million in 2020. However, it is estimated that only avout half of this energy consumption is used for effective processing, with the remaining 50% being consumed by idle servers. Serverless computing could significantly mitigate this inefficiency as there is an imminent need to develop strategies focused on energy and power management. Architectural design techniques for Serverless and FaaS applications play a crucial role in optimizing energy consumption and efficiency. These techniques involve various aspects, including task scheduling, resource allocation, workload consolidation, and load balancing. By maximizing resource utilization and minimizing energy waste, these strategies contribute to enhanced efficiency and sustainability in Serverless computing environments.

### 2.1 Serverledge

The approach proposed in this paper is implemented and evaluated in Serverledge. Serverledge[1] [24] is a FaaS framework designed to run serverless functions in the Edge-Cloud Continuum. Serverledge relies on multiple distributed computing nodes, organized in Edge and Cloud regions, based on proximity. Differently from the most popular FaaS frameworks designed for Cloud or clustered environments, Serverledge adopts a decentralized architecture, where each node can independently schedule and serve invocation requests with minimal interaction with the rest of the system. When overloaded, nodes can offload invocation requests to other nodes in their neighborhood (e.g., within Edge zones) or in higher layers of the infrastructure (e.g., from Edge to Cloud).

Functions in Serverledge are executed within isolated Docker containers, which are created and initialized on a "per-request" basis [16]. Similarly to most the existing frameworks, containers are re-used when possible to reduce initialization delays (i.e., cold starts).

The high-level architecture of Serverledge, depicted in Figure 1, consists of one or more *nodes* deployed either in Cloud data centers or at the network edge, along with a *global registry*, which provides distributed nodes with the required data about the system. Nodes are categorized into different *cloud regions* and *edge zones* based on their geographical location. Additionally, each Cloud region may incorporate a load balancer responsible for distributing incoming requests among nodes within the region.

A Serverledge node comprises a few key components, as outlined in Figure 2: API server, Scheduler, Local Registry, Offloader and Container Pool. The *API Server* interacts with clients as well as other nodes within the Serverledge network. The *scheduler* component collaborates with the *offloader* and *container pool* to schedule

[1] https://github.com/grussorusso/serverledge

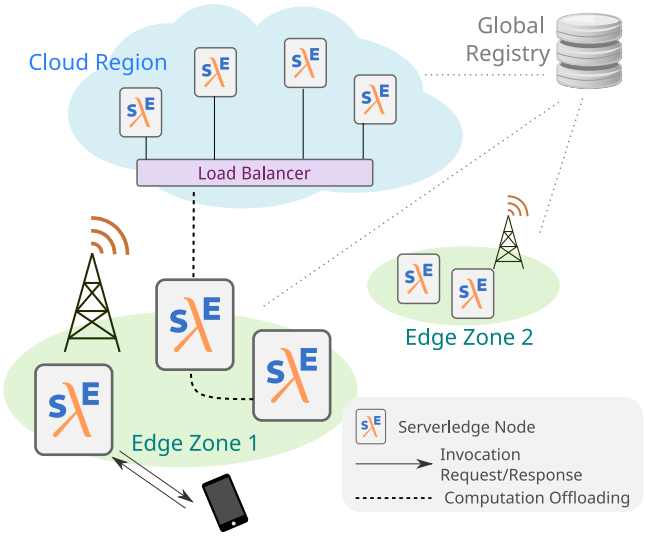



**Figure 1: Overview of Serverledge.**

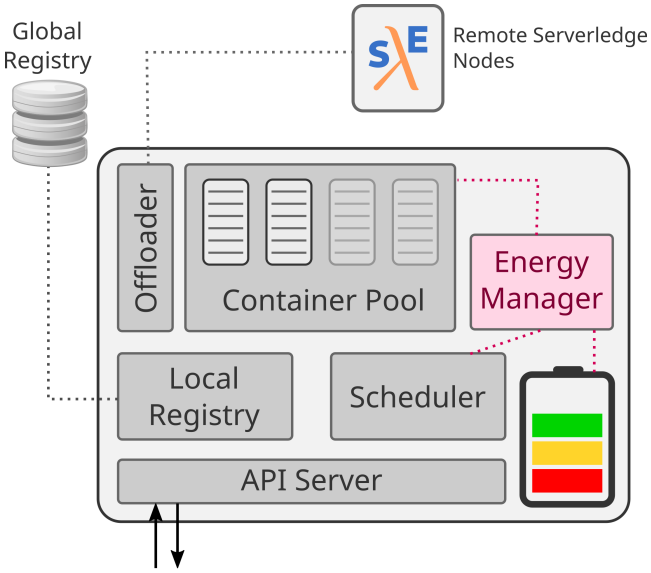



**Figure 2: Architecture of a Serverledge node extended with new energy-related monitoring and control components.**

tasks. Both system information collection and storing are locally managed by the *local registry*. Each Edge node conducts periodic monitoring tasks to collect data on neighboring nodes within the same Edge zone. Additionally, it implements the Vivaldi algorithm [6] to establish and maintain a virtual coordinate space, facilitating estimation of network distances between nodes through regular message exchanges. By interacting with each other and possibly with the *global registry* by its communication with the *local registry*, these components support execution and scheduling functionalities.

## 3 PROPOSED SOLUTION

In this section, we present our approach for energy-aware execution and offloading of serverless functions. First, we explain how we extend the architecture of Serverledge to integrate energy-related components and functionalities. Then, we describe the novel energy-oriented mechanisms in detail.

### 3.1 Serverledge Extension

As depicted in Figure 2, our Serverledge node architecture was extended to incorporate an additional energy-related component, named Energy Manager. The *Energy Manager* component oversees energy-related aspects, including monitoring of current energy availability and consumption. Specifically, in this work we consider FaaS nodes possibly powered by batteries. The Energy Manager monitors battery status and provides such information to the Scheduler component, which can use energy-related data to schedule incoming requests.

To monitor energy consumption, our choice gravitated towards RAPL [7]. *RAPL (Running Average Power Limit)* is a feature introduced by Intel to enable monitoring and control of processor power consumption. RAPL provide measures at the package, core, and memory levels. It is commonly used to manage power consumption in systems and can be accessed on Linux through the kernel powercap interface. RAPL is widely adopted for energy measurement and modeling as it provides estimated energy measurements for CPUs, integrated GPU, and DRAM. One way to check the power consumption using RAPL on Linux is reading the values directly from the `sysfs` interface. Specifically, to monitor power usage over time, we used the `energy_uj` file in the RAPL sysfs[2]. The file reports cumulative energy consumption (in $\mu J$) for the corresponding RAPL domain since the last reset of the RAPL counters, which usually happens at boot or after specific events.

Our goal is estimating the amount of energy drawn from the battery when executing serverless functions. To this end, we periodically retrieve updated RAPL measurements, and compute energy consumption in every time window. Specifically, the Energy Manager component runs a dedicated thread that regularly retrieves the amount of energy consumed by the node from RAPL every 60 seconds. The thread keeps track of the latest values retrieved from RAPL, so that, having obtained a new measurement, it computes the energy consumption of the last 1-minute window by subtracting the two latest observations.

Another innovative aspect focuses on the offloading process. In our vision, function execution offloading plays a key role in scenarios where a node is unable to handle a function invocation request due to low battery level, or serving the request would require an excessive amount of power compared to the benefits of running at the Edge. In such instances, the offloading mechanism can redirect the request to another Serverledge node with enough battery capacity left. By doing so, we effectively extend the node lifetime. Therefore, we aim to provide Serverledge scheduling component with essential insights into the energy status of nodes, complementing the existing data to facilitate decision-making during offloading operations.

Edge nodes in Serverledge run the Vivaldi [6] algorithm to create and update a network coordinates system at run-time. The resulting

[2]The file is located in `/sys/class/powercap/intel-rapl/intel-rapl:0`.

coordinates can be used, e.g., to select offloading target nodes based on proximity. The Vivaldi algorithm requires a continuous exchange of messages among Edge nodes. We exploited these messages to incorporate additional energy-related information. Specifically, we include the state of charge (SoC) of the battery in each message sent by every node, along along with other metrics already used (e.g., resource availability).

## 3.2 Adaptive Function Execution

Seeking to improve the energy efficiency of a Serverless system, we explored the idea of adaptive function execution, based on the energy status of the processing node and the idea of approximate computing. The idea behind approximate computing is that in many scenarios, exact calculations requires substantial resources, while a certain degree of approximation is acceptable. Allowing for some degree of approximation can greatly boost performance and save energy, while still maintaining acceptable results in terms of accuracy. Approximate computing represents an energy-efficient design approach for digital systems [11], which finds its most suitable applications in scenarios where computation inaccuracy is tolerated, such as multimedia content processing (e.g., audio, video, images), recognition tasks, and data mining. Thus, it can be effectively used in applications where an approximate result is sufficient for their purpose.

Our novel mechanism relies on the idea of *function variants*, where, for a given serverless function $f$, multiple implementation or configurations $f_1, f_2, \ldots, f_n$ exist. Each variant comes with its own resource and energy footprint. The concept of function variants, inspired by approximate computing, aims to reduce computational accuracy in order to save energy.

The adaptation strategy we design aims to prioritize the execution of function variants that consume less energy when the system faces energy constraints. It is expected that the execution of the variant with the lowest energy impact $f^*$ will result in longer battery lifetime.

In this work, we implement this idea by including additional information in the *context* data that is attached to every invocation request. Specifically, we include a flag that indicates whether the light function variant should be executed. We assume that function source code has been suitably modified to execute the proper variant depending on this flag.

## 3.3 Energy-Aware Policy

The novel mechanisms introduced in Serverledge to reduce energy consumption revolve around computational offloading and function variants. In this paper we devise a simple energy-aware control policy to exploit the aforementioned mechanisms and optimize the lifespan of battery-powered FaaS nodes. For simplicity, but without loss of generality, in this work we assume that two function variants exist for every function in the system. Specifically, given a baseline function implementation $f$, we consider an approximate and lighter implementation $f'$, for every function.

The novel scheduling policy introduced in Serverledge is described in Algorithm 1. To ensure the exploitation of both offloading and function variants, we establish two thresholds: $\text{thr}_{\text{Off}}$ and $\text{thr}_{\text{Var}}$, having $\text{thr}_{\text{Off}} < \text{thr}_{\text{Var}}$. The former represents the battery level at which the scheduler offloads computation of incoming requests to another FaaS node, while the latter indicates the battery level at which the system switches to executing the less energy-demanding function variant. These threshold values aim to respectively represent the circumstances in which a device either has a low battery or is approaching such condition.

**Algorithm 1** Energy-aware scheduling and offloading policy

**Require:** $SoC \in [0, 1]$ ▹ Remaining battery charge (%)
1: **if** $SoC > thr_{\text{Var}}$ **then**
2: execute $f$ locally
3: **else if** $SoC > thr_{\text{Off}}$ **then**
4: execute $f'$ locally
5: **else**
6: **if** $\exists$ neighbor with $SoC > thr_{\text{Off}}$ **then**
7: offload request
8: **else**
9: offload request to cloud
10: **end if**
11: **end if**

Our energy-aware policy guarantees that a Serverledge node executes a baseline function $f$ until the battery level is higher than the threshold $\text{thr}_{\text{Var}}$. Upon reaching this threshold, the lighter function variant $f'$ is executed locally. Subsequently, when the battery level hits the threshold $\text{thr}_{\text{Off}}$, the node starts offloading invocation requests to neighboring Serverledge nodes.

Specifically, as illustrated in Algorithm 1, upon the arrival of a function invocation request $f$, the Scheduler component retrieves the current battery level from the Energy Manager, indicated as $SoC$. This value is used to determine the scheduling action to take for the request. If the battery level exceeds $\text{thr}_{\text{Var}}$, the baseline implementation $f$ is executed. If $SoC$ is between $\text{thr}_{\text{Var}}$ and $\text{thr}_{\text{Off}}$, the function variant $f'$ is executed, leading to reduced energy consumption and consequently lower accuracy.

If the battery level is very low and – precisely – lower than $\text{thr}_{\text{Off}}$, offloading is attempted to avoid local execution. For this purpose, the Scheduler retrieves information from the Local Registry about neighboring nodes. If at least one node with enough remaining battery is available, the request is offloaded to it. If no suitable node is found, the request is offloaded to cloud.

# 4 EVALUATION

To evaluate the effectiveness of our solution, we perform experiments using our prototype implementation. We first describe the experimental setup we rely on, and then present the results we obtained.

## 4.1 Setup

**Infrastructure.** We deploy Serverledge in three CloudLab nodes [8]. Specifically, two nodes are used as Serverledge Edge nodes, and the third hosts a Serverledge Cloud node along with the Etcd-based Global Registry. We use the `c6320` machines of CloudLab, featuring an Intel Xeon 14-Core E5-2683V3 processor and 8 GB of memory. In the following, we will refer to the three Serverledge nodes as

node A (Edge node), node B (Edge node), and node C (Cloud node), for brevity.

**Emulated Battery.** As the used servers are not powered by batteries, we emulate a battery supply in our experiments. In particular, we adopt a simple approach, where: (i) we assume the battery to be fully charged at the beginning with a pre-defined capacity; (ii) at run-time, we update the remaining energy of the battery based on the real energy consumption we observe. Specifically, the configured battery capacity is 5.2 Ah and voltage is 3.7 V.

Upon startup of the Serverledge node, an energy monitoring thread is launched to periodically read RAPL measurements, as described in Sec. 3.1. These measurements are used to update the battery SoC. When the SoC of the battery decreases to 1% or less, the node shuts down.

**Workload.** In our experiments, we focus on studying the battery lifetime of node A. For this purpose, function invocation requests are all directed to node A. We consider 2 different serverless functions:

- **Euler**, which is a Python implementation of the Euler method for numerical resolution of ordinary differential equations [3]. When in energy-saving mode, we increase the *step size* parameter of the algorithm, trading off computational demand with approximation. Specifically, in our evaluation we use the explicit Euler method as an illustrative example. In this method, derivatives are estimated using finite differences as shown in (1), where $h$ is a discrete time step and $i$ is the iteration index.
$$\dot{x} = \frac{x_{i+1} - x_i}{h} \tag{1}$$
The threshold value $\text{thr}_{\text{Var}}$ is used to determine the value of the step size $h$, as follows:
$$h = \begin{cases} 10^{-5} & SoC > thr_{\text{Var}} \\ 10^{-3} & SoC \leq thr_{\text{Var}} \end{cases} \tag{2}$$
- **Image Classification**, implemented in Python, solves an image recognition task by means of convolutional neural networks (CNN). Given the URL of an image, the function retrieves the specified image and classifies it into one of the 1,000 classes used in the *ImageNet Large Scale Visual Recognition Challenge (ILSVRC)*[3]. For image classification, pre-trained models provided by Keras are used. Specifically, in the baseline configuration, the function relies on an ensemble classification model, which runs three different CNN models for inference: MobileNet, ResNet-50, and ResNet-152. When running the less energy-demanding variant, predictions are derived from the MobileNet model alone.

We select these functions as they allow us to easily define multiple variants with different computation-accuracy trade-offs.

To generate the workload, we use Locust[4], an open-source framework for load testing. Locust allows us to emulate the behavior of $N_U$ users concurrently issuing requests to Serverledge. Locust is deployed in a remote host compared to the Serverledge system.
In the Euler experiment we generate a maximum of $N_U$ = 20 concurrent users and with a ramp-up time of 0.3 user/sec. In Image Classification experiment, since this function requires more resources than the Euler one, we set $N_U$ = 10 concurrent users and ramp-up time equal to 0.03 user/sec. In both experiments, each user generates invocation requests at a constant throughput of 0.25 requests/second. Note that the generated workload is such to keep the Edge node highly loaded, with a fraction of the incoming requests being discarded in absence of computational resources.

**Serverledge Configuration.** We consider an energy-aware scenario, where we evaluate our approach, and a baseline scenario, where we use baseline Serverledge policies. In the former, nodes A and B run the energy-aware policy described in Sec. 3.3. In the latter, all the nodes rely on Serverledge's default scheduling policies [24], which greedily serve requests on the local host when enough computational resources are available. In our experiments, we run the two functions, Euler and Image Classification, under both baseline and energy-aware scenarios, to compare the results before and after implementing our novel energy-aware scheduling and offloading policy. By doing so, we aim to demonstrate the policy impact on system energy consumption.

Policy thresholds are set with the following configuration: $\text{thr}_{\text{Off}}$ = 20%, and $\text{thr}_{\text{Var}}$ = 40%. We consider, respectively, 20% as a commonly recognized threshold for low battery notifications on many devices, while 40% charge level as an acceptable tradeoff for starting to slow the battery consumption rate, because slightly below half of full charge, by triggering the system to execute the lowest energy impact function variant when $\text{thr}_{\text{Var}}$ reaches that threshold.

## 4.2 Results

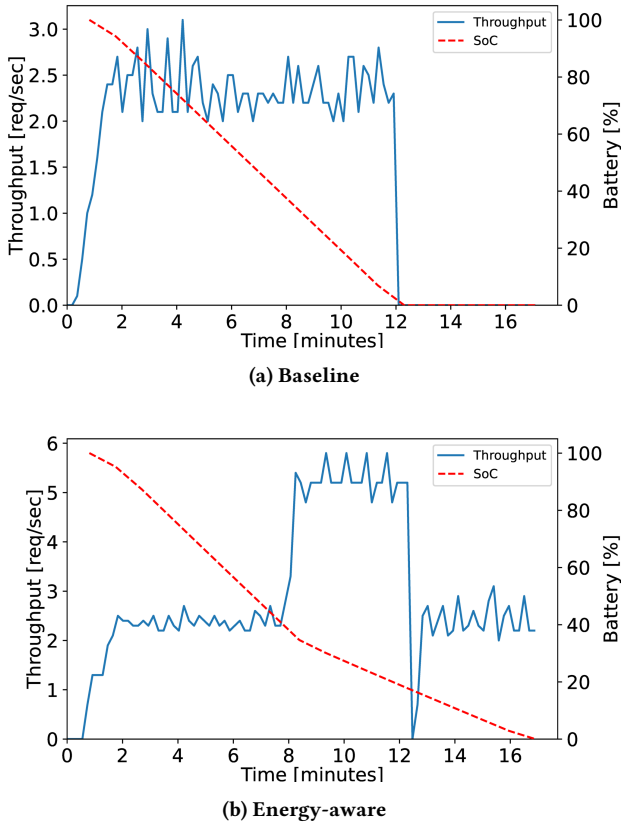
Figures 3 and 4 show throughput and battery SoC of node A over time in the performed experiments, until the node runs out of energy and shuts down.

Figure 3a displays the results when executing the baseline Euler function. The experiment runs for a duration of 12 minutes until the node battery is depleted, completing a total of 1,615 requests. It can be observed that the rate of battery discharge is almost constant, as no energy-aware policy is in use. Figure 3b shows the results for the alternative energy-aware configuration, where we use the novel mechanisms and policy we propose. In this case, the experiment duration is extended to 17 minutes, with a 40% increase of the node lifespan compared to the baseline scenario. The node manages to complete a total of 2,838 completed requests, 75% more than the baseline. Looking at the plot, we can note that energy consumption significantly decreases when the battery SoC becomes lower than the threshold $\text{thr}_{\text{Var}}$ and the lighter function variant starts being used. We also note that this function variant has a reduced resource demand and allows the node to complete requests more quickly, as demonstrated by the throughput increase.

Around minute 12, when the battery level hits the 20% threshold ($\text{thr}_{\text{Off}}$), the node starts offloading all incoming requests to other Serverledge nodes. This results in a drop in throughput due to cold starts of containers in Edge B node. Once started, we observe a similar throughput as measured in the first phase of the experiment, approximately 2.5 requests per second.

The behavior of the system is similar when considering the Image Classification function, with the associated results reported in Figure 4. Specifically, Figure 4a shows the results in the baseline

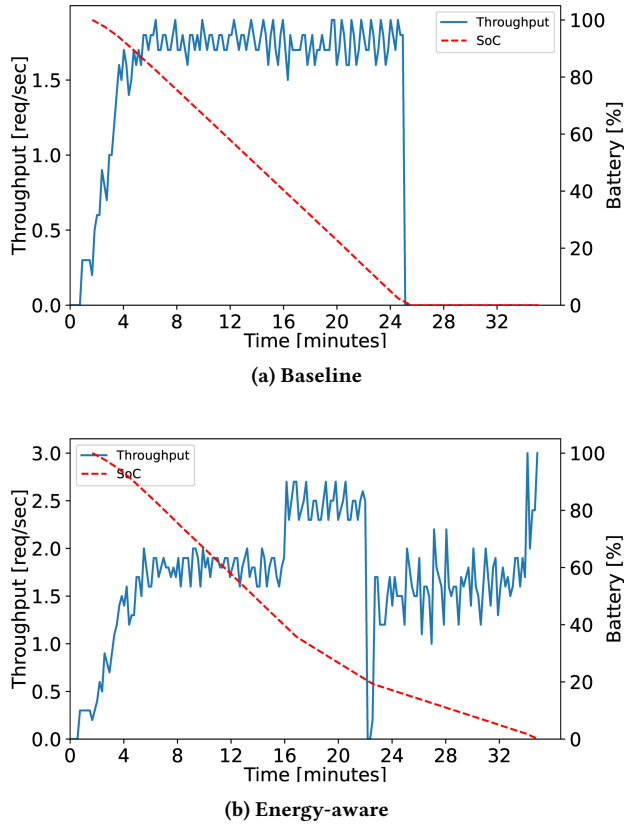
[3]https://www.image-net.org/challenges/LSVRC/index.php
[4]https://locust.io/




(a) Baseline

(b) Energy-aware

**Figure 3: Experiments with the Euler function.**




(a) Baseline

(b) Energy-aware

**Figure 4: Experiments with the Image Classification function.**

scenario. The experiment lasts 25 minutes until the processing node battery is drained, completing a total of 2,356 requests. Figure 4b showcases the energy-aware execution of the Image Classification function. The battery lifetime is prolonged to 35 minutes, reflecting a 10-minute increase. This also results in a significant increase in the completed requests, i.e., 3,475 in this energy-aware configuration.

Observing the rate of battery depletion, we can note a consistent behavior as described for the Euler function. It is interesting to further observe that, at the end of the energy-aware experiment, in this case we note a second throughput increase. This behavior is caused by the fact that battery of Edge node B reaches the threshold $thr_{Var}$ as well, and starts executing the lighter variant of the function.

A summary of the results is given in Figure 5, which shows the number of completed invocation requests in each experimental scenario. The figure also shows a breakdown of the completed requests, highlighting the number of completions associated with the baseline implementation, the lighter variant, and the offloaded requests. This visualization showcases the effectiveness of the energy-aware policy in prolonging the lifespan and enhancing the throughput of battery-powered nodes through energy-aware offloading and execution strategies.

## 5 RELATED WORK

The attention of the research community on sustainability issues of serverless programming model and underlying computing systems has recently grown [17, 26], with the goal to make energy efficiency a primary consideration in designing and managing such systems. The serverless programming and execution model, which does not require the computing equipment to be active all the time, opens up new control knobs that can be explored in an agile manner to effectively reduce the carbon footprint of computing. These knobs range from scheduling functions in a carbon and energy-aware manner, also taking the reduction of energy-hungry cold starts into account, to offloading and migrating functions to energy-friendly locations, to trading off the quality of the function computed results with its energy footprint. On the other hand, we note that well-known serverless frameworks, such as OpenFaaS and OpenWhisk, and Cloud services, such as AWS Lambda and Azure Functions, are primarily designed to be energy-agnostic. The same observation applies to research prototypes designed for Faas at the Edge and that are closest to Serverledge, such as Colony [15], Faasm [27], and tinyFaaS [18].

So far, most works in the sustainable serverless field studied how to place and schedule function invocations in an energy-efficient

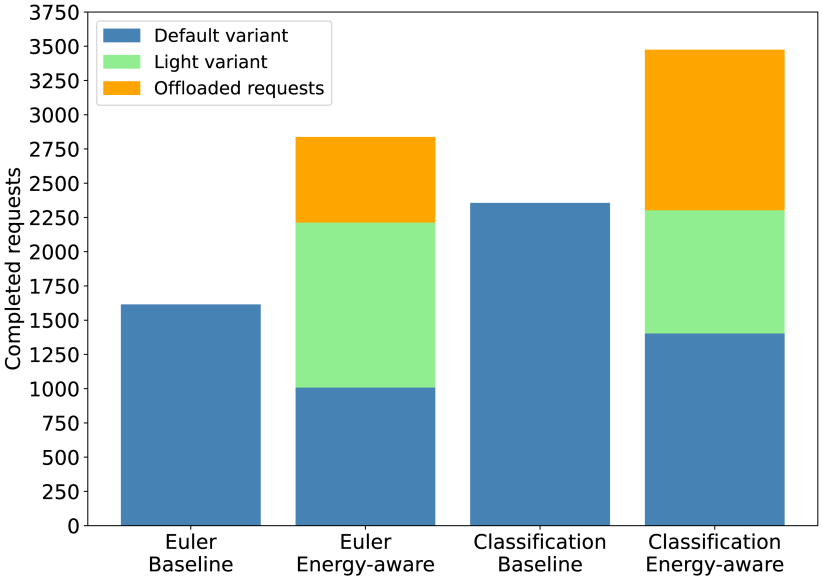



**Figure 5: Completed requests in the experiments.**

way [1, 2, 5, 20, 28], ranging from locally clustered to edge computing environments. Tsenos et al. [28] proposed an energy-efficient scheduler for functions executed in a cluster environment with the goal to minimize energy consumption while meeting each function's performance requirements. To this end, their scheduler places the function containers in the cluster based on various metrics or historical data, the function service level objectives, and workload type. *GreenCourier* [5] is a Kubernetes-based scheduling framework that enables the runtime scheduling of serverless functions across geographically distributed regions based on carbon efficiency. *EneX* [20] is an energy-aware serverless scheduler which is based on an optimization problem formulation to assign chains of functions having predefined deadlines with the goal of minimizing energy consumption in a clustered environment. *FaasHouse* [1] is an energy-aware scheduler for serverless edge systems, which exploits edge nodes to allow function execution near the users. It considers battery-operated edge nodes powered by renewable energy and relies on an imbalanced energy-minimizing problem to assign functions to edge nodes. Similarly to FaasHouse, we also exploit offloading to move functions away from low-powered nodes.

With respect to these works, our work complements their schedulers by adding the energy-aware adaptive function execution and offloading control mechanisms, which can be integrated with an energy-efficient scheduler to devise a serverless system that fully manages energy as a first-class resource in the compute continumm scenario.

Another control knob that can be exploited for energy efficiency in the field of serverless computing is the reduction of cold starts. Towards this end, *ATOM* [10] utilizes a deep reinforcement learning model to predict cold starts. Such a control knob can be effectively integrated with the ones we propose in this work.

Research efforts focused also on the exploitation of heterogeneous devices to execute serverless functions in an energy-efficient manner. For instance, within the *Function Delivery Network* proposed in [12], a scheduling system for invocation requests selects the device to run functions trading-off various metrics, including energy consumption. In this landscape, resource heterogeneity is also a consequence of heterogeneous power supplies (e.g., battery-powered devices, devices connected to renewable power supplies, conventional data centers). This kind of scenario is studied by Aslanpour et al. [2], who target battery-operated and renewable-energy powered nodes. They devise scheduling algorithms to improve the operational availability of nodes.

A different approach is represented by *MicroFaaS* [4], a FaaS framework designed and optimized to run on bare-metal, ARM single-board computers, rather than traditional virtualization-based cloud servers. The proposed system is shown to achieve significant energy savings compared to server-based systems.

To the best of our knowledge, the idea of exploiting multiple, alternative function implementations of serverless functions to trade-off among conflicting objectives has been used only in *xAFCL* [21]. The authors design a serverless platform to deploy and schedule function choreographies in a multi-cloud scenario. They allow users to deploy multiple implementations of each function, with the system determining the one to execute at run-time based on the current objectives (e.g., makespan, cost). Energy consumption is not considered though.

## 6 CONCLUSION

In this paper we presented an approach for energy-aware execution of serverless functions in an Edge-Cloud environment. Our solution is inspired by the approximate computing paradigm, and leverages multiple implementations of each function to trade-off energy consumption and computation accuracy as needed. Computational offloading is also exploited to move function execution from the Edge to the Cloud, to extend the lifespan of battery-powered edge devices. We implemented the approach in Serverledge, an open-source FaaS framework, and demonstrated its benefits through experiments.

For future work, we plan to extend and improve our solution, addressing some key issues. First, in this work we emulated battery discharge, as the hosts used in the experiments were not equipped with physical batteries. We will consider using different devices to perform experiments with real batteries. Furthermore, our approach assumes that function source code is modified to support the execution of multiple variants. We will investigate strategies to simplify the adoption of our approach by automatically tweaking function source code and generating new variants, as well as adding an option to disable energy-aware execution if a function has a strict high accuracy constraint. Finally, we plan to extend the QoS-aware offloading policies proposed in [23] with energy-awareness.

## ACKNOWLEDGEMENTS

This work has been partially supported by the Spoke 1 "FutureHPC & BigData" of the Italian Research Center on High-Performance Computing, Big Data and Quantum Computing (ICSC) funded by MUR Missione 4 Componente 2 Investimento 1.4: Potenziamento strutture di ricerca e creazione di "campioni nazionali" di R&S (M4C2-19) - Next Generation EU (NGEU).

## REFERENCES

[1] M. Aslanpour, A. N. Toosi, M. Cheema, and M. Chhetri. 2024. faasHouse: Sustainable Serverless Edge Computing through Energy-aware Resource Scheduling. *IEEE Trans. Serv. Comput.* (2024), 1–14. https://doi.org/10.1109/TSC.2024.3354296

[2] M. S. Aslanpour, A. N. Toosi, M. A. Cheema, and R. Gaire. 2022. Energy-Aware Resource Scheduling for Serverless Edge Computing. In *Proc. of 22nd IEEE Int'l Symp. on Cluster, Cloud and Internet Computing, CCGrid '22*. IEEE, 190–199. https://doi.org/10.1109/CCGrid54584.2022.00028

[3] J. C. Butcher. 2003. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons.

[4] A. Byrne, Y. Pang, A. Zou, S. Nadgowda, and A. K. Coskun. 2022. MicroFaaS: Energy-efficient Serverless on Bare-metal Single-board Computers. In *Proc. of 2022 Design, Automation & Test in Europe Conference & Exhibition, DATE '22*. IEEE, 754–759. https://doi.org/10.23919/DATE54114.2022.9774688

[5] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud. 2023. GreenCourier: Carbon-Aware Scheduling for Serverless Functions. In *Proc. of 9th Int'l Workshop on Serverless Computing, WoSC '23*. ACM, 18–23. https://doi.org/10.1145/3631295.3631396

[6] R. Cox, F. Dabek, M. F. Kaashoek, J. Li, and R. Tappan Morris. 2004. Practical, Distributed Network Coordinates. *ACM SIGCOMM Comput. Commun. Rev.* 34, 1 (2004), 113–118. https://doi.org/10.1145/972374.972394

[7] H. David, E. Gorbatov, U. R. Hanebutte, R. Khanna, and C. Le. 2010. RAPL: Memory Power Estimation and Capping. In *Proc. of 2010 ACM/IEEE Int'l Symp. on Low-Power Electronics and Design, ISLPED '10*. IEEE, 189–194. https://doi.org/10.1145/1840845.1840883

[8] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, et al. 2019. The Design and Operation of CloudLab. In *Proc. of 2019 USENIX Ann. Tech. Conf., ATC '19*. 1–14. https://www.flux.utah.edu/paper/duplyakin-atc19

[9] P. K. Gadepalli, S. McBride, G. Peach, L. Cherkasova, and G. Parmer. 2020. Sledge: A Serverless-first, Light-weight Wasm Runtime for the Edge. In *Proc. of ACM/IFIP Middleware 2020*. ACM, 265–279. https://doi.org/10.1145/3423211.3425680

[10] M. Golec, S. S. Gill, F. Cuadrado, A. K. Parlikad, M. Xu, H. Wu, and S. Uhlig. 2023. ATOM: AI-Powered Sustainable Resource Management for Serverless Edge Computing Environments. *IEEE Trans. Sustain. Comput.* (2023), 1–13. https://doi.org/10.1109/TSUSC.2023.3348157

[11] J. Han and M. Orshansky. 2013. Approximate Computing: An Emerging Paradigm for Energy-Efficient Design. In *Proc. of 18th IEEE European Test Symp., ETS '13*. IEEE, 1–6. https://doi.org/10.1109/ETS.2013.6569370

[12] A. Jindal, M. Gerndt, M. Chadha, V. Podolskiy, and P. Chen. 2021. Function Delivery Network: Extending Serverless Computing for Heterogeneous Platforms. *Softw. Pract. Exp.* 51, 9 (2021), 1936–1963. https://doi.org/10.1002/spe.2966

[13] V. Kjorveziroski, S. Filiposka, and V. Trajkovik. 2021. IoT Serverless Computing at the Edge: A Systematic Mapping Review. *Comput.* 10, 10 (2021), 130. https://doi.org/10.3390/COMPUTERS10100130

[14] S. Kounev, N. Herbst, C. L. Abad, A. Iosup, Ian T. Foster, P. J. Shenoy, O. F. Rana, and A. A. Chien. 2023. Serverless Computing: What It Is, and What It Is Not? *Commun. ACM* 66, 9 (2023), 80–92. https://doi.org/10.1145/3587249

[15] F. Lordan, D. Lezzi, and R. M. Badia. 2021. Colony: Parallel Functions as a Service on the Cloud-Edge Continuum. In *Proc. of 27th Int'l Conf. on Parallel and Distributed Computing, Euro-Par '21 (LNCS, Vol. 12820)*. Springer, 269–284. https://doi.org/10.1007/978-3-030-85665-6_17

[16] N. Mahmoudi and H. Khazaei. 2022. Performance Modeling of Serverless Computing Platforms. *IEEE Trans. Cloud Comput.* 10, 4 (2022), 2834–2847. https://doi.org/10.1109/TCC.2020.3033373

[17] P. Patros, J. Spillner, A. Papadopoulos, B. Varghese, O. Rana, and S. Dustdar. 2021. Toward Sustainable Serverless Computing. *IEEE Internet Comput.* 25 (2021), 42–50. https://doi.org/10.1109/MIC.2021.3093105

[18] T. Pfandzelter and D. Bermbach. 2020. tinyFaaS: A Lightweight FaaS Platform for Edge Environments. In *Proc. of 2020 IEEE Int'l Conference on Fog Computing, ICFC '20*. IEEE, 17–24. https://doi.org/10.1109/ICFC49376.2020.00011

[19] P. Raith, S. Nastic, and S. Dustdar. 2023. Serverless Edge Computing - Where We Are and What Lies Ahead. *IEEE Internet Comput.* 27, 3 (2023), 50–64. https://doi.org/10.1109/MIC.2023.3260939

[20] S. H. Rastegar, H. Shafiei, and A. Khonsari. 2024. EneX: An Energy-Aware Execution Scheduler for Serverless Computing. *IEEE Transactions on Industrial Informatics* 20, 2 (2024), 2342–2353. https://doi.org/10.1109/TII.2023.3290985

[21] S. Ristov, S. Pedratscher, and T. Fahringer. 2023. xAFCL: Run Scalable Function Choreographies Across Multiple FaaS Systems. *IEEE Trans. Serv. Comput.* 16, 1 (2023), 711–723. https://doi.org/10.1109/TSC.2021.3128137

[22] G. Russo Russo, V. Cardellini, and F. Lo Presti. 2023. Serverless Functions in the Cloud-Edge Continuum: Challenges and Opportunities. In *Proc. of 2023 31st Euromicro Int'l Conf. on Parallel, Distributed and Network-Based Processing, PDP '23'*. IEEE, 321–328. https://doi.org/10.1109/PDP59025.2023.00056

[23] G. Russo Russo, D. Ferrarelli, D. Pasquali, V. Cardellini, and F. Lo Presti. 2024. QoS-aware Offloading Policies for Serverless Functions in the Cloud-to-Edge Continuum. *Future Gener. Comput. Syst.* 156 (2024), 1–15. https://doi.org/10.1016/j.future.2024.02.019

[24] G. Russo Russo, T. Mannucci, V. Cardellini, and F. Lo Presti. 2023. Serverledge: Decentralized Function-as-a-Service for the Edge-Cloud Continuum. In *Proc. of 2023 IEEE Int'l Conference on Pervasive Computing and Communications, PerCom '23*. IEEE, 131–140. https://doi.org/10.1109/PERCOM56429.2023.10099372

[25] J. Schleier-Smith, V. Sreekanti, A. Khandelwal, J. Carreira, N. Jayant Yadwadkar, R. Ada Popa, J. E. Gonzalez, I. Stoica, and D. A. Patterson. 2021. What Serverless Computing is and Should Become: The Next Phase of Cloud Computing. *Commun. ACM* 64, 5 (2021), 76–84. https://doi.org/10.1145/3406011

[26] P. Sharma. 2023. Challenges and Opportunities in Sustainable Serverless Computing. *SIGENERGY Energy Inform. Rev.* 3, 3 (2023), 53––58. https://doi.org/10.1145/3630614.3630624

[27] S. Shillaker and P. Pietzuch. 2020. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *Proc. of 2020 USENIX Ann. Tech. Conf., ATC '20*. USENIX Association, 419–433. https://www.usenix.org/system/files/atc20-shillaker.pdf

[28] M. Tsenos, A. Peri, and V. Kalogeraki. 2023. Energy Efficient Scheduling for Serverless Systems. In *Proc. of 4th IEEE Int'l Conference on Autonomic Computing and Self-Organizing Systems, ACSOS '23*. IEEE Computer Society, 27–36. https://doi.org/10.1109/ACSOS58161.2023.00020

[29] R. Xie, Q. Tang, S. Qiao, H. Zhu, F. R. Yu, and T. Huang. 2021. When Serverless Computing Meets Edge Computing: Architecture, Challenges, and Open Issues. *IEEE Wirel. Commun.* 28, 5 (2021), 126–133. https://doi.org/10.1109/MWC.001.2000466